

COMP333_PROJECT_GROUP_O

Course: COMP 333 — Final Project Phase 1

Group: O

Names:

- Carson Johnston - **Student ID:** 40312846
- Charlotte Lauzon - **Student ID:** 40285642
- Ava Samimi - **Student ID:** 40048117

Description of the task:

The objective of Phase 1 of this project is to select a large, real world dataset (≥ 1 GB). We then do the data retrieval, the wrangling, an EDA with 2 research questions, a baseline model and a report on Jupyter Notebook.

Source of the datasets:

The **US Used Cars** dataset is a public dataset available on Kaggle:

<https://www.kaggle.com/datasets/ananymital/us-used-cars-dataset>

Description of the datasets:

The **US Used Cars** dataset exceeds the minimum size requirements, has a large number of features, includes real world inconsistencies and supports both supervised and unsupervised learning tasks.

It contains detailed information about 3 million used cars details across the United States.

Each row represents a single vehicle listing, with each column being an attribute.

The *US Used Cars* dataset contains:

- Vehicle Identification (Vehicle identification number, listing ID)
- Vehicle Specifications (Engines, horsepower, fuel, size, legroom, bed, body...)
- Market/Dealer Information (city, ZIP, position, days on market, date of listing)
- Fuel efficiency
- Condition and History (accidents, damages, if it was a cab, if it is new...)
- Categorical and Descriptive Features (colours, description, picture...)

The dataset has a mix of numerical, boolean, high-cardinality categorical and string variables

The notebook will include:

1. Data Retrieval
2. Wrangling/Cleaning
3. EDA
4. Baseline Model
5. Report

Division-of-labour

The work was split and shared by all team members. More precisely:

Carson: Step 2 (Wrangling/Cleaning), with contributions to step 3 (Research question) and step 4 (evaluation, discussion)

Charlotte: Step 1 (Data Retrieval), with contributions to step 2 (cleaning pipeline), step 3 (Research question), step 4 (setup of the model, train/val/test, linear regression, metrics and evaluation setup) and the overall introduction and README.md

Ava: Step 3 (EDA), with contributions to step 4 (discussion)

```
#import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import matplotlib.gridspec as gridspec
from pandas.api.types import is_numeric_dtype, is_bool_dtype
import warnings
import os
import re

# Avoid unnecessary warnings in output
#warnings.filterwarnings("ignore")

# Doesn't cut tables
pd.set_option('display.width', 1000)
pd.set_option('display.max_colwidth', 50)
pd.set_option('display.max_columns', 30)

# Apply predefined visual theme for consistence in the export
plt.style.use('seaborn-v0_8')
sns.set_theme()
```

1. Data Retrieval: Phase 1 – Step 1

1.1 Source

We are using the **US Used Cars** public dataset from Kaggle:

<https://www.kaggle.com/datasets/ananymital/us-used-cars-dataset>

This dataset is downloaded as a zipped csv file, and, extracted, has a size of approximately 9.98 GB.

It contains tabular vehicle listing data.

1.2 Retrieval

The dataset is retrieved as a **file-based CSV source** and is loaded into Python using the pandas function `read_csv()`.

We selected file-based retrieval, because the data is structured and tabular, and Kaggle provides it as a flat CSV file.

Also, by retrieving the data as a CSV file, it avoids API authentication dependency. Since the dataset is static and doesn't change in real time, the most reproducible and efficient approach is to store the raw file locally.

1.3 Challenge Handling

- **Large file size (~10GB):** Because the Data file is nearly 10 GB, direct loading can be slow and memory-heavy.
 - Mitigation:
 - we use `low_memory=False`. This makes sure the entire file is read before deciding the columns' data type. This prevents dtype warnings.
 - For development purposes, we also used a 200k-row sampled subset of the data to speed up iteration and testing.
- **Data type Inconsistencies:** Some numeric variables are stored as strings; During the Data Wrangling Phase, these will be cleaned and converted.
- **Authentication / Rate Limits:** Not applicable in this phase because there are no API calls used (download is done once via Kaggle).
 - If Kaggle API were used, authentication would require a private Kaggle API token. However, for this phase, one static file download was more efficient and safe. To ensure grading reproducibility, we avoid credential-based retrieval,.

1.4 Raw Data Storage

The dataset is stored in the project's root directory and is treated as read-only to preserve the data integrity. All transformations and cleaning steps are performed on the copied dataset.

```
# DO NOT LOAD THIS UNLESS YOU HAVE A VERY POWERFUL COMPUTER
# This is to load the full 9.98GB dataset
'''
```

```

raw_data_path = "used_cars_data.csv"

# Load the raw dataset
if os.path.exists(raw_data_path):
    print("Loading full dataset...")
    cars = pd.read_csv(raw_data_path, low_memory=False)
else:
    print("Full dataset not found. Loading sample dataset...")
    cars = pd.read_csv("sample_used_cars_data.csv", low_memory=False)

print("Shape:", cars.shape)
cars.head()
'''

```

```

'\nraw_data_path = "used_cars_data.csv"\n\n# Load the raw dataset\nif
os.path.exists(raw_data_path):\n    print("Loading full dataset...")\n    cars =
pd.read_csv(raw_data_path, low_memory=False)\nelse:\n    print("Full dataset not
found. Loading sample dataset...")\n    cars =
pd.read_csv("sample_used_cars_data.csv", low_memory=False)\n\nprint("Shape:",
cars.shape)\ncars.head()\n'

```

```

# TEST DATASET
# For testing purposes, load this subset

raw_data_path = "used_cars_data.csv"

# Load the raw dataset
if os.path.exists(raw_data_path):
    print("Loading full dataset for testing purposes...")
    testing_cars = pd.read_csv(raw_data_path, nrows=200000, low_memory=False)
else:
    print("Full dataset not found. Loading sample dataset...")
    testing_cars = pd.read_csv("sample_used_cars_data.csv",nrows=200000,
low_memory=False)

print("Shape:", testing_cars.shape)
testing_cars.head()

```

Full dataset not found. Loading sample dataset...
Shape: (200000, 66)

	vin	back_legroom	bed	bed_height	bed_length	body_type	cabin	city
0	ZACNJABB5KPJ92081	35.1 in	NaN	NaN	NaN	SUV / Crossover	NaN	Bayamon

	vin	back_legroom	bed	bed_height	bed_length	body_type	cabin	city
1	SALCJ2FX1LH858117	38.1 in	NaN	NaN	NaN	SUV / Crossover	NaN	San Juan
2	JF1VA2M67G9829723	35.4 in	NaN	NaN	NaN	Sedan	NaN	Guaynab
3	SALRR2RV0L2433391	37.6 in	NaN	NaN	NaN	SUV / Crossover	NaN	San Juan
4	SALCJ2FXXLH862327	38.1 in	NaN	NaN	NaN	SUV / Crossover	NaN	San Juan

5 rows x 66 columns

```
# JUST FOR EXPORTING THE SAMPLE CSV
# Export the DataFrame to a new CSV file
# index=False prevents pandas from writing the row numbers as an extra column
#output_file_path = 'test_cars.csv'
#testing_cars.to_csv(output_file_path, index=False)
```

2. Wrangling/Cleaning: Phase 1 – Step 2

We perform an initial data audit:

- Missing value analysis
- Duplicate detection
- Outlier detection
- Data type validation
- Reproducible cleaning pipeline

We use the `quantDDA()` and `vizDDA()` functions from Lab 2 to standardize our audit process.

2.1 INITIAL AUDIT

```
# The function quantDDA(df) takes a pandas DataFrame as argument
# The function returns a new DataFrame with a summary of each column
def quantDDA(df):

# create a list to put all column's summary in
summary_list = []
```

```

# Loop through all columns of the argument dataset
for column in df.columns:
    # One column is a series, we create a dictionary to store the statistics
    series = df[column]
    summary = {}

    # Store the column name as the feature
    summary["Feature"] = column
    # Store the number of rows of the database
    summary["Number of Observations"] = len(series)
    # Store the number of only valid, non-missing values
    summary["Number of Entries"] = series.count()
    # Store the number of distinct values only
    summary["Number of Unique Entries"] = series.nunique()
    # Store the number of missing values (where its n/a is "true")
    summary["Number of Missing Entries"] = series.isna().sum()

    # Store the mode values as a list
    modes = series.mode()
    summary["Mode(s)"] = list(modes)

    # The categorical columns do not have all the numeric fields;
    # To make sure the table has no issues with columns, we set as NaN
    summary["Number of Outliers (IQR)"] = np.nan
    summary["Number of Extreme Values (top/bottom 1%)"] = np.nan
    summary["Mean"] = np.nan
    summary["Standard Deviation"] = np.nan
    summary["Maximum"] = np.nan
    summary["Minimum"] = np.nan
    summary["Q1"] = np.nan
    summary["Q2 (Median)"] = np.nan
    summary["Q3"] = np.nan
    summary["Skewness"] = np.nan
    summary["Kurtosis"] = np.nan

    # If its numerical column, compute numeric statistics
    # Otherwise, skip the calculations
    if pd.api.types.is_numeric_dtype(series) and not
pd.api.types.is_bool_dtype(series):
        # We remove missing values to not compute with invalid numbers
        # pandas ignore NaN, but not SciPy, so we drop beforehand
        x = series.dropna()

        # Store all the numerical statistics
        if len(x) > 0:
            summary["Mean"] = x.mean()
            summary["Standard Deviation"] = x.std()
            summary["Maximum"] = x.max()
            summary["Minimum"] = x.min()
            summary["Q1"] = x.quantile(0.25)

```

```

summary["Q2 (Median)"] = x.median()
summary["Q3"] = x.quantile(0.75)

# Number of outliers with IQR Method
IQR = summary["Q3"] - summary["Q1"]
lower = summary["Q1"] - 1.5 * IQR
upper = summary["Q3"] + 1.5 * IQR
# Count values outside the lower and upper bounds
outliers = x[(x < lower) | (x > upper)]
summary["Number of Outliers (IQR)"] = outliers.count()

# Extreme values (Top and Bottom 1%)
lower_ext = x.quantile(0.01)
upper_ext = x.quantile(0.99)
# Counts the number of extreme values of the top and bottom 1%
extreme = x[(x < lower_ext) | (x > upper_ext)]
summary["Number of Extreme Values (top/bottom 1%)"] = extreme.count()

# Use SciPy for skewness and kurtosis
summary["Skewness"] = stats.skew(x, bias=False)
summary["Kurtosis"] = stats.kurtosis(x, fisher=True, bias=False)

```

```

# Add the dictionary to list
summary_list.append(summary)

```

```

result = pd.DataFrame(summary_list)

```

```

# Make sure the columns are in the same order as in assignment
ordered_columns = [

```

```

    "Feature",
    "Number of Observations",
    "Number of Entries",
    "Number of Unique Entries",
    "Number of Missing Entries",
    "Number of Outliers (IQR)",
    "Number of Extreme Values (top/bottom 1%)",
    "Mode(s)",
    "Mean",
    "Standard Deviation",
    "Maximum",
    "Minimum",
    "Q1",
    "Q2 (Median)",
    "Q3",
    "Skewness",
    "Kurtosis"

```

```

]

```

```

return result[ordered_columns]

```

```

# The function vizDDA(df) takes a pandas DataFrame as argument
# The function produces a square grid of plots (univariate on diagonal, bivariate off
diagonal),
# and a heatmap of the missing values.
def vizDDA(df):

    df = df.copy()

# Sample if too large to avoid slow crosstabs and memory issues
    max_rows = 5000
    if len(df) > max_rows:
        df = df.sample(n=max_rows, random_state=42)

# Remove high-cardinality categorical columns for visualization grid
    high_cardinality_cols = []

    for col in df.columns:
        if not pd.api.types.is_numeric_dtype(df[col]):
            # put an arbitrary 20 threshold
            if df[col].nunique() > 20:
                high_cardinality_cols.append(col)
# df for the plots
    df_plot = df.drop(columns=high_cardinality_cols)

# Limit grid size to avoid memory/speed issues (max 6x6)
    max_features = 6
    if len(df_plot.columns) > max_features:
        df_plot = df_plot.iloc[:, :max_features]
    num_features = len(df_plot.columns)

# Detect datetime columns
    is_datetime = {
        column: pd.api.types.is_datetime64_any_dtype(df_plot[column])
        for column in df_plot.columns
    }

# Create square grid
# put 4x4 inch space for each subplot for readability
    fig = plt.figure(figsize=(4 * num_features, 4 * num_features))
    grid = gridspec.GridSpec(num_features, num_features)

# Consider i as index for rows, and j as index for columns
    for i, col_i in enumerate(df_plot.columns):
        for j, col_j in enumerate(df_plot.columns):

            ax = fig.add_subplot(grid[i, j])

            # Detect if its numeric or categorical
            is_num_i = pd.api.types.is_numeric_dtype(df_plot[col_i])

```

```

is_num_j = pd.api.types.is_numeric_dtype(df_plot[col_j])

# Univariate on diagonal
# if we compare a feature with itself, we are doing a univariate analysis
if i == j:

    # If it's a datetime univariate, we count how many times each unique
datetime appear
    if is_datetime[col_i]:
        series = df_plot[col_i].dropna()
        if len(series) > 0:
            series.value_counts().sort_index().plot(ax=ax)
            ax.set_ylabel("count")

    # if numeric, we do a histogram to visualize the distribution
    elif is_num_i:
        sns.histplot(df_plot[col_i], kde=True, ax=ax)

    # For categorical univariate, we do a bar chart
    else:
        df_plot[col_i].value_counts().plot(kind="bar", ax=ax)

# Bivariate off diagonal
# We study two different variables
else:

    # If one variable is datetime, and the other is numeric:
    # We do a line plot
    # Datetime vs Numeric when datetime is the row variable, and numeric
is column variable
    if is_datetime[col_i] and is_num_j:
        # Select the relevant columns, drop the missing values, and sort
by the datetime column
        tmp = df_plot[[col_i, col_j]].dropna().sort_values(col_i)
        # We draw datetime on x-axis, numeric on y-axis
        ax.plot(tmp[col_i], tmp[col_j])

    # Same but for the symmetric situation;
    # Datetime vs Numeric when datetime is the column variable, and
numeric is row variable
    elif is_datetime[col_j] and is_num_i:
        tmp = df_plot[[col_j, col_i]].dropna().sort_values(col_j)
        ax.plot(tmp[col_j], tmp[col_i])

    # If both variables are numeric (Numeric vs numeric), we do a scatter
plot
    elif is_num_i and is_num_j:
        sns.scatterplot(x=df_plot[col_j], y=df_plot[col_i], ax=ax)

    # If there are mixed types (categorical vs numeric) we do a boxplot

```

```

# For when numerical is the row feature and categorical is the column
feature

elif is_num_i and not is_num_j and not is_datetime[col_j]:
    tmp = df_plot[[col_j, col_i]].dropna()
    tmp[col_j] = tmp[col_j].astype("category")
    sns.boxplot(data=tmp, x=col_j, y=col_i, ax=ax)

# For when numerical is the column feature and categorical is the row
feature

elif not is_num_i and is_num_j and not is_datetime[col_i]:
    tmp = df_plot[[col_i, col_j]].dropna()
    tmp[col_i] = tmp[col_i].astype("category")
    sns.boxplot(data=tmp, x=col_i, y=col_j, ax=ax)

# If both variable are categorical (Categorical vs categorical), we
do a grouped bar chart
# Since all cases with numerical variables have been checked,
# only categorical on both row and column left
else:
    categorical = pd.crosstab(df_plot[col_i], df_plot[col_j])
    categorical.plot(kind="bar", ax=ax)

# Axis labeling
# Only label bottom row with x labels
if i == num_features - 1:
    ax.set_xlabel(col_j, fontsize=15, fontweight="bold")
else:
    ax.set_xlabel("")

# Only label left column with y labels
if j == 0:
    ax.set_ylabel(col_i, fontsize=15, fontweight="bold")
else:
    ax.set_ylabel("")

# Add title
fig.suptitle("Visualization Grid (Univariate on Diagonal, Bivariate Off-
Diagonal)", fontsize=25, fontweight="bold")
# Improve spacing so subplots don't overlap
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

# Heatmap of missing values
plt.figure(figsize=(12, 6))
sns.heatmap(df.isna(), cbar=True, vmin=0, vmax=1)
plt.title("Missing Values Heatmap")
plt.show()

```

General Findings of the First Audit

```
quantDDA(testing_cars)
```

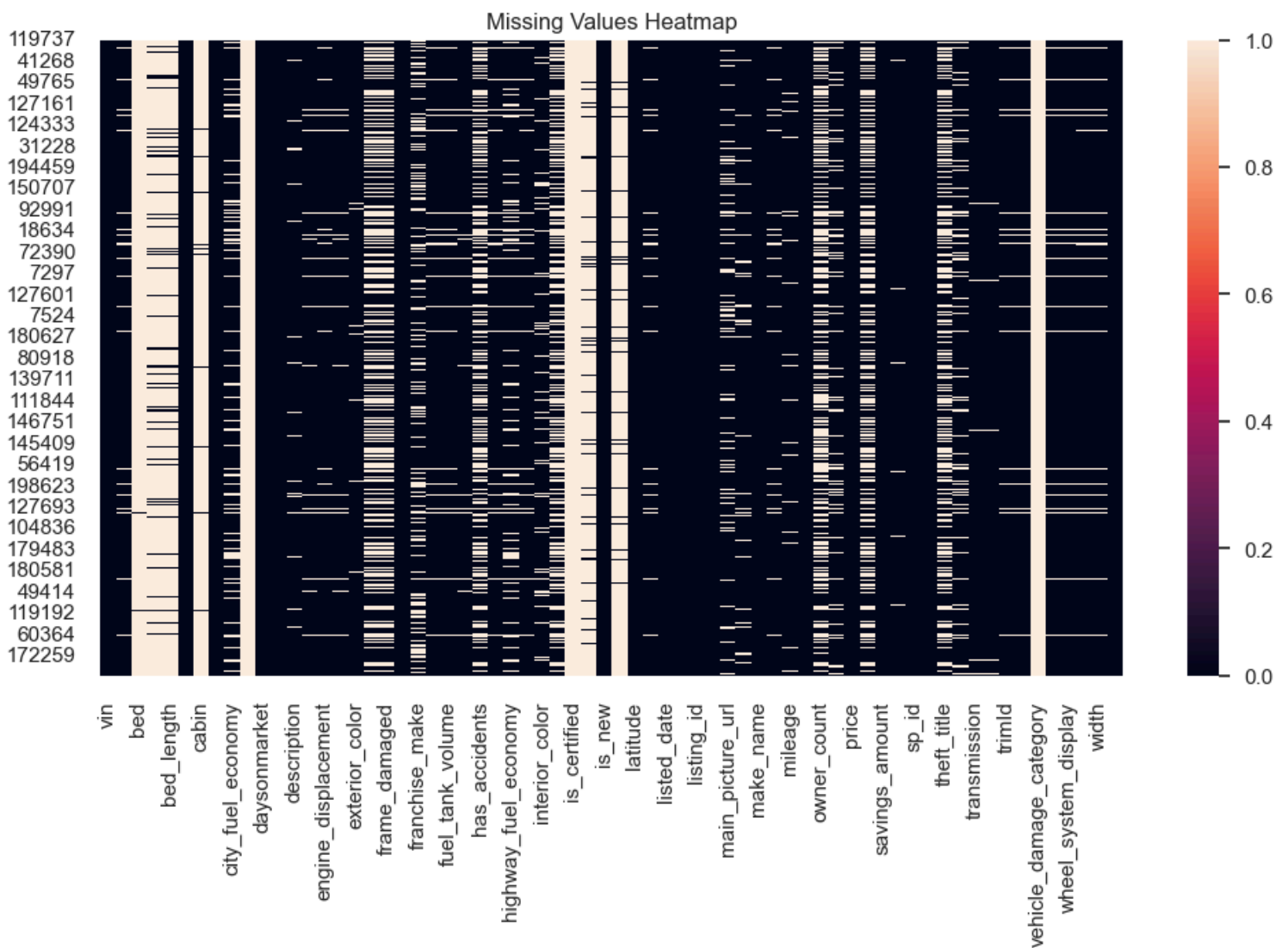
	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top/bottom 1%)	Mode(s)
0	vin	200000	200000	200000	0	NaN	NaN	[00000000 00000000 0000000..]
1	back_legroom	200000	190542	196	9458	NaN	NaN	[38.3 in]
2	bed	200000	1205	3	198795	NaN	NaN	[Short]
3	bed_height	200000	20252	1	179748	NaN	NaN	[--]
4	bed_length	200000	20252	68	179748	NaN	NaN	[67.1 in]
...	...	...	...	...	...	...	...	...
61	wheel_system	200000	191406	5	8594	NaN	NaN	[AWD]
62	wheel_system_display	200000	191406	5	8594	NaN	NaN	[All-Whee
63	wheelbase	200000	190542	422	9458	NaN	NaN	[106.3 in]
64	width	200000	190542	266	9458	NaN	NaN	[73 in]
65	year	200000	200000	72	0	19150.0	1618.0	[2020]

66 rows x 17 columns

```
vizDDA(testing_cars)
```

Visualization Grid (Univariate on Diagonal, Bivariate Off-Diagonal)





2.2: Reproducible Pipeline

For the cleaning pipeline, we first manually remove columns. Then, we apply a function to clean the dataset

A cleaning Report is provided for an easier view of what was performed.

Manual Removal of Columns

Due to time constraints, computing costs and the scope of this project. Below is a method that manually removes columns from the large dataframe. The columns selected to be removed were chosen by doing data analysis with functions found below such as `clean_cars()`. The reason `remove_cars_columns()` is implemented so early on is to speed up the computing times of processes below. This function only works on the "used_cars_data.csv" file used for this assignment. It will not work with other data sets.

The idea of this is to reduce the amount of columns in the data frame so more rows can be loaded in for better data accuracy and better performance

The manual removal of columns is meant to compliment the reproducible cleaning pipeline. It is not a replacement and will not remove columns that our pipeline does automatically.

Removal of Copy Columns

This process involves finding columns that share similar data and removing them. For example "wheel_system" and "wheel_system_display" both describe if cars are front wheel drive (FWD) or all wheel drive (AWD). "wheel_system_display" is not needed and will be removed from the data frame.

Removal of Descriptive Columns

This process involves removing columns that focus on describing car features, rather than giving simple data. It is hard to process descriptive data, so columns such as "description" will be removed.

Removal of Unecessary Columns

Remove columns that don't add much value to the data analysis. These columns are removed to allow for more rows from more useful columns. 'trimId' is an example of a column that doesn't provide much usefulness.

```
def remove_cars_columns(df):
    copy_columns = ['wheel_system_display', 'engine_cylinders', 'latitude',
                    'longitude', 'exterior_color', 'listing_id', 'power']
    descriptive_columns = ['description', 'trim_name', 'transmission_display',
                           'sp_name', 'major_options']
    unnecessary_columns = ['trimId', 'dealer_zip', 'savings_amount', 'sp_id',
                           'seller_rating']

    remove_columns = []
    remove_columns.extend(copy_columns)
    remove_columns.extend(descriptive_columns)
    remove_columns.extend(unnecessary_columns)

    return df.drop(columns=[c for c in remove_columns if c in df.columns],
                   errors='ignore')

testing_cars = remove_cars_columns(testing_cars)
```

Cleaning of the Data Set

Reproducible Data Cleaning Pipeline

The clean_cars() function implements a structured and reproducible data cleaning pipeline. Compared to the manual removal process described earlier, this function performs automated transformations based on measurable data properties (e.g., missingness rate, data type detection, regex pattern matching).

This function ensures that cleaning steps are systematic, transparent, and reproducible.

Defensive Copy of the Dataset

A copy of the dataframe is created to prevent accidental modification of the original dataset. This preserves the raw data as read-only.

Dropping Columns with Extremely High Missingness

This step computes the proportion of missing values per column, removes columns where the missing rate exceeds a user-defined threshold, and stores removed columns in a cleaning report.

We do this because columns with excessive missing values add noise, increase computational cost, and provide unreliable signal for modeling.

Removing Duplicate Rows

This step removes fully duplicated rows, because duplicates can bias statistical results, artificially inflate dataset size, and distort frequency distributions.

The number of removed rows is tracked in the report for transparency.

Handling Missing Values (Numeric)

For numeric columns, missing values are imputed using a missing indicator column and the median. This preserves information about missingness via a flag, and median imputation is robust to outliers.

Handling Missing Values (Categorical)

For categorical columns, missing values are replaced with a label "Missing". This preserves row count, avoids dropping potentially useful data, and allows models to treat the missingness as a category.

String Unification and Measurement Normalization

We standardize string formatting by converting to string, removing the whitespace and detecting measurements columns. For example, we checked if a column contained a pattern such as "20 in", "15.5 in"...

If the pattern was detected, we extracted only the numeric portion, and converted to float. We also renamed to column to reflect the units.

```
#Create function for pipeline
def clean_cars(df, drop_missing_threshold):
    df = df.copy()

    report = {}

    # Drop columns with extremely high missingness
    missing_rate = df.isna().mean()
    drop_cols = missing_rate[missing_rate > drop_missing_threshold].index.tolist()
    df = df.drop(columns=drop_cols)
    report["dropped_cols_high_missing"] = drop_cols

    # Remove row duplicates
```

```

before = len(df)
df = df.drop_duplicates()
report["rows_removed_duplicates"] = before - len(df)

# Missing values
# Numeric columns: add missing flag
imputed_numeric = []
for col in df.columns:
    if is_numeric_dtype(df[col]) and not is_bool_dtype(df[col]):
        if df[col].isna().any():
            df[f"{col}_missing"] = df[col].isna()
            df[col] = df[col].fillna(df[col].median())
            imputed_numeric.append(col)
report["numeric_imputed_with_flags"] = imputed_numeric

# For categorical columns: fill missing with label
filled_categorical = []
for col in df.columns:
    if df[col].dtype == "object" and df[col].isna().any():
        df[col] = df[col].fillna("Missing")
        filled_categorical.append(col)
report["categorical_filled"] = filled_categorical

# String unification & converting inches measurements into numeric values
for col in df.select_dtypes(include=["object", "string"]).columns:
    # Standardize strings first
    df[col] = df[col].astype(str).str.strip()

    # Check if the column mostly contains "number in" patterns
    # We'll check the first non-null value as a heuristic
    sample_value = df[col].dropna().iloc[0] if not df[col].empty else ""

    # Regex pattern: look for digits (and optional decimal) followed by 'in'
    if re.search(r'\d+\.\?\d*\s*in', str(sample_value).lower()):
        # 1. Extract the number
        df[col] = df[col].str.extract(r'(\d+\.\?\d*)').astype(float)

        # 2. Rename the column (e.g., 'wheel_size' -> 'wheel_size_in')
        df = df.rename(columns={col: f"{col}_in"})

# column "fuel_tank_volume" will be converted to numerical data
# Ensure the column is treated as a string
column_as_string = df['fuel_tank_volume'].astype(str)

# Extract the numbers (including decimals) using Regex
# (\d+\.\?\d*) looks for digits, an optional dot, and more digits
df['fuel_tank_volume'] = column_as_string.str.extract(r'(\d+\.\?\d*)').astype(float)

# Rename the column to keep track of the units

```

```

df = df.rename(columns={'fuel_tank_volume': 'fuel_tank_gal'})

# column "maximum_seating" will be converted to numerical data
# ensure teh column is treated as a string
column_as_string = df['maximum_seating'].astype(str)

# extract the numbers using Regex
# (\d+\.\d*) looks for digits, an optional dot, and more digits. I have no idea
if vehicles can have half seats
df['maximum_seating'] = column_as_string.str.extract(r'(\d+\.\d*)').astype(float)

return df, report

```

Cleaning Report

```

cars_clean, cleaning_report = clean_cars(testing_cars, 0.1)
cleaning_report

```

```

{'dropped_cols_high_missing': ['bed',
 'bed_height',
 'bed_length',
 'cabin',
 'city_fuel_economy',
 'combine_fuel_economy',
 'fleet',
 'frame_damaged',
 'franchise_make',
 'has_accidents',
 'highway_fuel_economy',
 'interior_color',
 'isCab',
 'is_certified',
 'is_cpo',
 'is_oemcpo',
 'main_picture_url',
 'owner_count',
 'salvage',
 'theft_title',
 'torque',
 'vehicle_damage_category'],
 'rows_removed_duplicates': 0,
 'numeric_imputed_with_flags': ['engine_displacement',
 'horsepower',
 'mileage'],
 'categorical_filled': ['back_legroom',

```

```
'body_type',  
'engine_type',  
'front_legroom',  
'fuel_tank_volume',  
'fuel_type',  
'height',  
'length',  
'maximum_seating',  
'transmission',  
'wheel_system',  
'wheelbase',  
'width']}]}
```

Preview of the Cleaned Dataset

```
cars_clean.head(10)
```

	vin	back_legroom_in	body_type	city	daysonmarket	engine_displaceme
0	ZACNJABB5KPJ92081	35.1	SUV / Crossover	Bayamon	522	1300.0
1	SALCJ2FX1LH858117	38.1	SUV / Crossover	San Juan	207	2000.0
2	JF1VA2M67G9829723	35.4	Sedan	Guaynabo	1233	2500.0
3	SALRR2RV0L2433391	37.6	SUV / Crossover	San Juan	196	3000.0
4	SALCJ2FXXLH862327	38.1	SUV / Crossover	San Juan	137	2000.0
5	SALYK2EX1LA261711	37.1	SUV / Crossover	San Juan	242	2000.0
6	3MZBPABL6KM107908	35.1	Sedan	Bayamon	447	2500.0
7	SALYK2EX5LA275434	37.1	SUV / Crossover	San Juan	70	2000.0
8	SALCJ2FX6LH858128	38.1	SUV / Crossover	San Juan	196	2000.0
9	SALZL2GX4LH007593	33.8	SUV / Crossover	San Juan	510	2000.0

Summary Statistics for the cleaned dataset

Removal of outliers

The function to remove outliers will only be implemented at the end of the EDA and data cleaning. This is to prevent functions like quantDDA from doing analysis on data that has outliers removed.

The process of removing outliers relies on simple algorithms like the one found in quantDDA, but also requires a human understanding of the data columns. For example, the quantDDA function might identify 0 as an outlier for the "mileage" column, but it is very reasonable for a new car to have not driven.

For the outlier removal function a list will be created for columns that require different observations. Just below is the list in markdown format with reasoning for special treatment.

- maximum_seating: Between 2 and 15 seats is reasonable for small cars and vans
- mileage: Cars with 0 mileage are possible if they are brand new. Only upper outliers will be removed
- year: Only exclude the lower end of outliers. Cars made the year the data was collected are not outliers.

IMPORTANT: only use this function once data cleaning is final and you are ready to move on to proper analysis.

```
# remove outliers specific to used cars dataset
def remove_car_outliers(df):
    # list of columns to completely ignore, just add column names to ignore for
    # testing
    columns_ignore = ['maximum_seating']

    for column in df.columns:
        series = df[column]
        # only compute numeric types
        if pd.api.types.is_numeric_dtype(series) and not
pd.api.types.is_bool_dtype(series):
            # only act on columns that need outliers removed
            if column not in columns_ignore:

                summary = {}

                Q1 = series.quantile(0.25)
                Q3 = series.quantile(0.75)
                IQR = Q3 - Q1

                lower = Q1 - 1.5 * IQR
                upper = Q3 + 1.5 * IQR

                # only remove lower outliers for year
                if column == 'year':
                    df = df[df[column] >= lower]
                elif column == 'mileage':
                    df = df[df[column] <= upper]
                else:
                    # Remove both upper and lower outliers for any other included
                    # numeric columns
                    df = df[(df[column] >= lower) & (df[column] <= upper)]
```

```
return df
```

```
quantDDA(cars_clean)
```

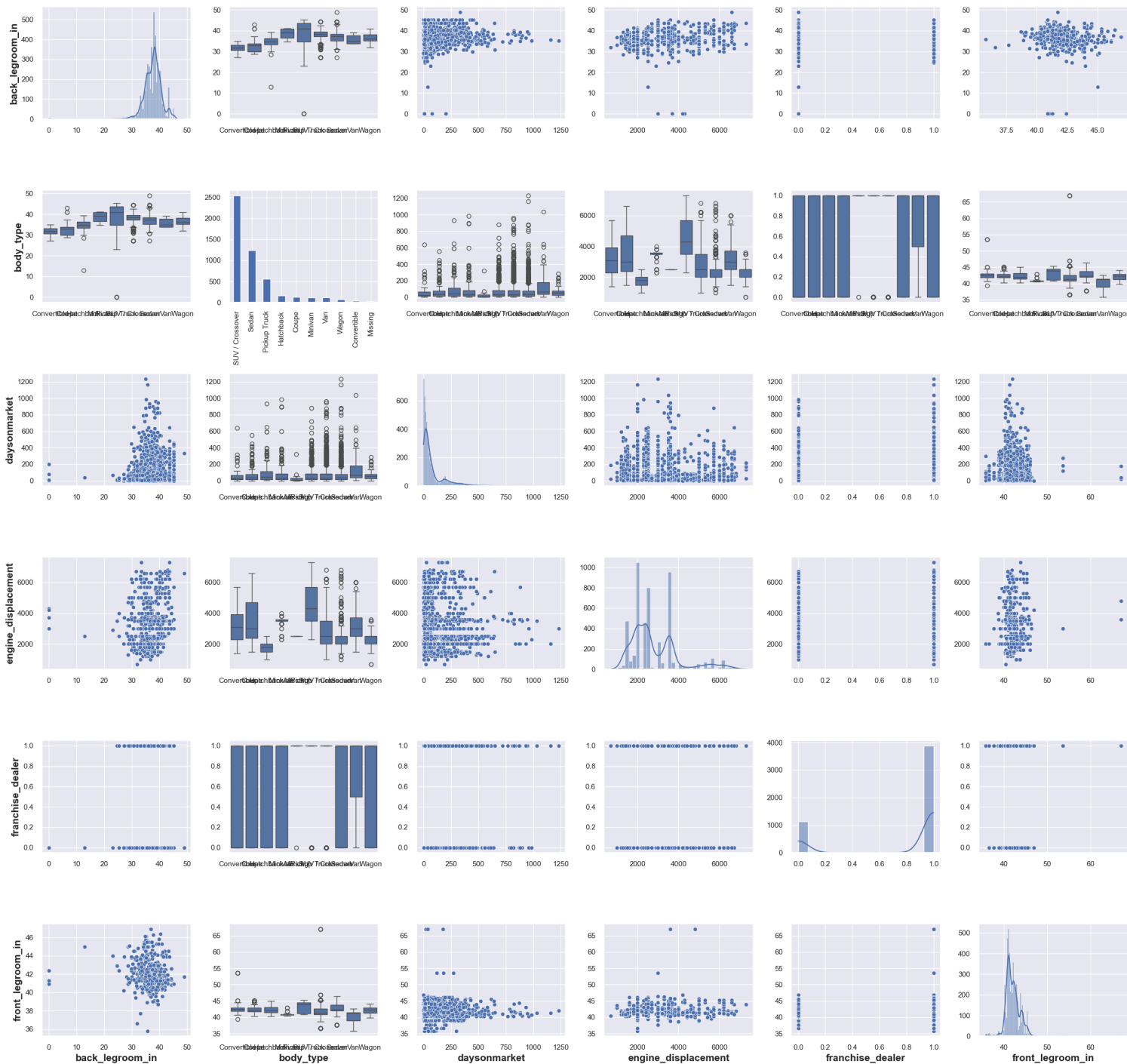
	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top/bottom 1%)	M
0	vin	200000	200000	200000	0	NaN	NaN	[000000]
1	back_legroom_in	200000	183561	195	16439	4637.0	3591.0	[300000]
2	body_type	200000	200000	10	0	NaN	NaN	[S000000]
3	city	200000	200000	2146	0	NaN	NaN	[N000000]
4	daysonmarket	200000	200000	1000	0	28794.0	1994.0	[600000]
5	engine_displacement	200000	200000	65	0	7229.0	2795.0	[200000]
6	engine_type	200000	200000	32	0	NaN	NaN	[I400000]
7	franchise_dealer	200000	200000	2	0	NaN	NaN	[T000000]
8	front_legroom_in	200000	189042	87	10958	2101.0	3554.0	[400000]
9	fuel_tank_gal	200000	190435	166	9565	5575.0	2475.0	[100000]
10	fuel_type	200000	200000	8	0	NaN	NaN	[G000000]
11	height_in	200000	190505	440	9495	1358.0	3715.0	[600000]
12	horsepower	200000	200000	413	0	2138.0	3650.0	[200000]
13	is_new	200000	200000	2	0	NaN	NaN	[F000000]
14	length_in	200000	190506	753	9494	21262.0	3688.0	[100000]
15	listed_date	200000	200000	1004	0	NaN	NaN	[200000]
16	listing_color	200000	200000	15	0	NaN	NaN	[B000000]
17	make_name	200000	200000	63	0	NaN	NaN	[F000000]
18	maximum_seating	200000	190502	11	9498	57116.0	757.0	[500000]
19	mileage	200000	200000	70214	0	15683.0	2000.0	[000000]
20	model_name	200000	200000	977	0	NaN	NaN	[F000000]
21	price	200000	200000	40426	0	6148.0	3934.0	[900000]
22	transmission	200000	200000	5	0	NaN	NaN	[A000000]
23	wheel_system	200000	200000	6	0	NaN	NaN	[A000000]
24	wheelbase_in	200000	190506	421	9494	20757.0	3686.0	[100000]
25	width_in	200000	190505	265	9495	1336.0	3397.0	[700000]
26	year	200000	200000	72	0	19150.0	1618.0	[200000]
27	engine_displacement_missing	200000	200000	2	0	NaN	NaN	[F000000]

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top/bottom 1%)	M
28	horsepower_missing	200000	200000	2	0	NaN	NaN	[F
29	mileage_missing	200000	200000	2	0	NaN	NaN	[F

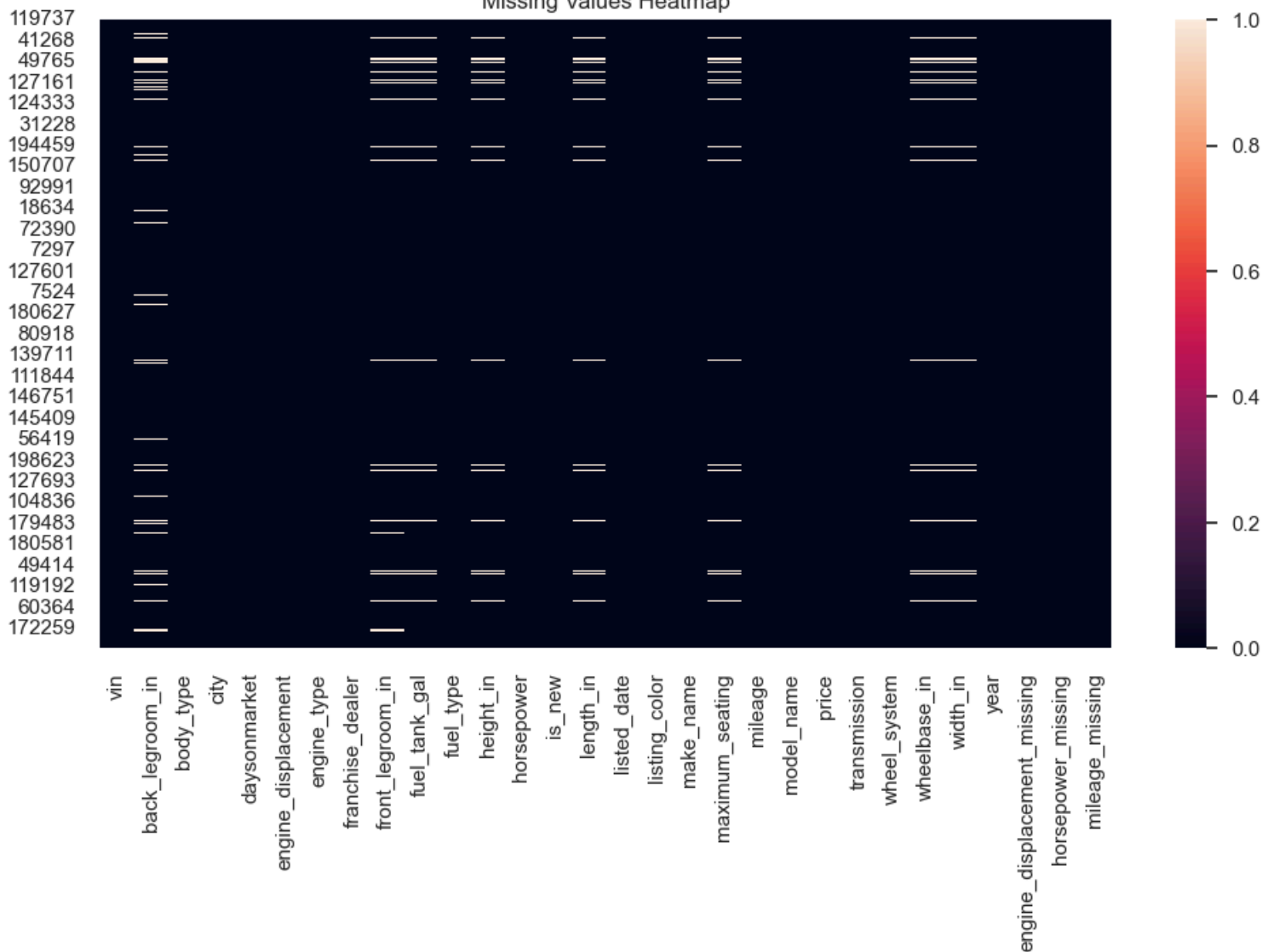
Visualizations for the cleaned dataset

```
try:
    vizDDA(cars_clean)
except NameError:
    vizDDA(testing_cars) # Fallback if cars_clean not yet created
```

Visualization Grid (Univariate on Diagonal, Bivariate Off-Diagonal)



Missing Values Heatmap



```
# remove the outliers
cars_clean = remove_car_outliers(cars_clean)
quantDDA(cars_clean)
```

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top/bottom 1%)	M
0	vin	113002	113002	113002	0	NaN	NaN	[19, 19, 19]
1	back_legroom_in	113002	113002	110	0	970.0	1565.0	[3, 3, 3]
2	body_type	113002	113002	8	0	NaN	NaN	[S, S, S]
3	city	113002	113002	1666	0	NaN	NaN	[N, N, N]
4	daysonmarket	113002	113002	183	0	6364.0	1080.0	[6, 6, 6]
5	engine_displacement	113002	113002	31	0	1370.0	946.0	[2, 2, 2]
6	engine_type	113002	113002	18	0	NaN	NaN	[I4, I4, I4]

	Feature	Number of Observations	Number of Entries	Number of Unique Entries	Number of Missing Entries	Number of Outliers (IQR)	Number of Extreme Values (top/bottom 1%)	M
7	franchise_dealer	113002	113002	2	0	NaN	NaN	[T
8	front_legroom_in	113002	113002	57	0	1860.0	1107.0	[4
9	fuel_tank_gal	113002	113002	102	0	845.0	1836.0	[1
10	fuel_type	113002	113002	5	0	NaN	NaN	[G
11	height_in	113002	113002	188	0	0.0	1688.0	[6
12	horsepower	113002	113002	194	0	106.0	1618.0	[1
13	is_new	113002	113002	2	0	NaN	NaN	[F
14	length_in	113002	113002	309	0	560.0	1481.0	[1
15	listed_date	113002	113002	184	0	NaN	NaN	[2
16	listing_color	113002	113002	15	0	NaN	NaN	[B
17	make_name	113002	113002	34	0	NaN	NaN	[H
18	maximum_seating	113002	113002	6	0	21325.0	1076.0	[5
19	mileage	113002	113002	43935	0	3232.0	1130.0	[0
20	model_name	113002	113002	375	0	NaN	NaN	[E
21	price	113002	113002	25283	0	981.0	2261.0	[1
22	transmission	113002	113002	5	0	NaN	NaN	[A
23	wheel_system	113002	113002	6	0	NaN	NaN	[A
24	wheelbase_in	113002	113002	147	0	0.0	1144.0	[1
25	width_in	113002	113002	176	0	0.0	1547.0	[7
26	year	113002	113002	9	0	0.0	0.0	[2
27	engine_displacement_missing	113002	113002	2	0	NaN	NaN	[F
28	horsepower_missing	113002	113002	2	0	NaN	NaN	[F
29	mileage_missing	113002	113002	2	0	NaN	NaN	[F

Run the cell below first if you get **NameError** — it loads the data for Part 3.

3. Exploratory Data Analysis (EDA)

We perform exploratory data analysis on the cleaned dataset to understand distributions, relationships between variables, and to formulate our research questions.

3.1 Summary Statistics

We compute summary statistics for numerical and categorical variables to characterize the central tendency, spread, and composition of the data.

3.2 Univariate Visualizations

We visualize the distribution of key variables individually: price, body type, year, and mileage.

3.3 Bivariate Visualizations

We examine relationships between pairs of variables, particularly how price relates to mileage, body type, and year.

3.4 Correlation Analysis

We compute and visualize the correlation matrix for numerical features to identify linear relationships that may inform modeling.

3.5 EDA Synthesis: Comparative Analysis

We synthesize findings from 3.1–3.4 to identify consistent patterns and bridge to our research questions.

3.6 Research Questions

SUPERVISED LEARNING — RESEARCH QUESTION 1

Can we predict the selling price of a used vehicle given its specifications, usage history, and dealership characteristics?

- **Y (target variable):** `price` — the listed or selling price of the vehicle (continuous, in dollars).
- **X (input features):**
 - **Specifications:** `year`, `mileage`, `engine_displacement`, `horsepower`, `body_type`, `engine_type`, `wheel_system`, `transmission`, `fuel_type`, `back_legroom_in`, `front_legroom_in`, `height_in`, `width_in`, `wheelbase_in`
 - **Usage / condition:** `mileage` (odometer reading), `daysonmarket`, `is_new`
 - **Dealership:** `franchise_dealer`, `city`

This is a **regression** task: we aim to estimate the continuous price value from the feature vector X. A baseline model (e.g., linear regression) will predict Y from X.

UNSUPERVISED LEARNING — RESEARCH QUESTION 2

Can we discover natural clusters of used vehicles (e.g., budget, mid-range, luxury) based on their specifications and market characteristics?

- **X (input features used for clustering):**
 - **Specifications:** `engine_displacement`, `horsepower`, `body_type`, `year`, `wheel_system`, `fuel_type`
 - **Market / economic:** `price`, `mileage`, `daysonmarket`
 - **Size / comfort:** `back_legroom_in`, `front_legroom_in`, `width_in`, `height_in`
- **Y (output):** Cluster labels (e.g., 0, 1, 2) — **discovered, not given**. Each label corresponds to a group such as budget (lower price, smaller engine, higher mileage), mid-range, or luxury (higher price, larger

engine, lower mileage). No ground-truth labels exist; the model learns the groupings from X.

This is a **clustering** task (e.g., K-means): we aim to partition vehicles into meaningful groups based on similarity in the feature space.

```
# === PART 3 SETUP: Ensure data is loaded (run this if you get NameError) ===
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

if globals().get('testing_cars') is None and globals().get('cars_clean') is None:
    base = os.getcwd()
    paths = [
        os.path.join(base, "used_cars_data.csv"),
        os.path.join(base, "data", "raw", "used_cars_data.csv"),
        "used_cars_data.csv",
    ]
    found = next((p for p in paths if os.path.exists(p)), None)
    if found:
        testing_cars = pd.read_csv(found, nrows=3000, low_memory=False)
        print("Loaded data from:", found, "| Shape:", testing_cars.shape)
    else:
        np.random.seed(42)
        testing_cars = pd.DataFrame({
            "price": np.random.uniform(15000, 70000, 3000), "mileage":
np.random.uniform(0, 150000, 3000),
            "year": np.random.randint(2012, 2023, 3000), "horsepower":
np.random.uniform(100, 400, 3000),
            "body_type": np.random.choice(["Sedan", "SUV", "Truck"], 3000),
            "make_name": np.random.choice(["Toyota", "Ford", "Honda"], 3000),
        })
        print("Using demo data (run cell 4 to load real CSV)")
    else:
        print("Data already loaded.")
```

Data already loaded.

3.1 Summary Statistics

```
# 3.1 Summary Statistics
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Use cars_clean or testing_cars - globals().get() never raises NameError
```

```

df = globals().get('cars_clean'); df = df if df is not None else
globals().get('testing_cars')
if df is None:
    raise NameError("Run cells 1-4 first (imports + data loading) to create
testing_cars.")

# Numeric columns (exclude _missing flags)
numeric_cols = [c for c in df.select_dtypes(include=[np.number]).columns if not
str(c).endswith('_missing')]

# Categorical summary
print("\nTop body types:")
print(df['body_type'].value_counts().head(10) if 'body_type' in df.columns else
"N/A")
print("\nTop makes:")
print(df['make_name'].value_counts().head(10) if 'make_name' in df.columns else
"N/A")

```

```

Top body types:
body_type
SUV / Crossover    71285
Sedan              33579
Minivan           2633
Hatchback         2575
Wagon             1623
Coupe              992
Convertible        240
Van                75
Name: count, dtype: int64

```

```

Top makes:
make_name
Honda            11028
Ford             10886
Chevrolet        10266
Toyota           10111
Jeep             8616
Nissan            8102
Subaru           6872
Hyundai          5461
Kia              4928
BMW              3644
Name: count, dtype: int64

```

Conclusion (3.1): The summary statistics and tables above show that price, mileage, and year vary widely across the dataset. Body types such as SUV/Crossover and Sedan dominate, while certain makes

(e.g., Honda, Ford, Chevrolet) appear most frequently. These patterns establish a baseline for understanding the market composition and guide which variables may be most predictive for modeling.

3.2 Univariate Visualizations

```
# 3.2 Univariate Visualizations
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

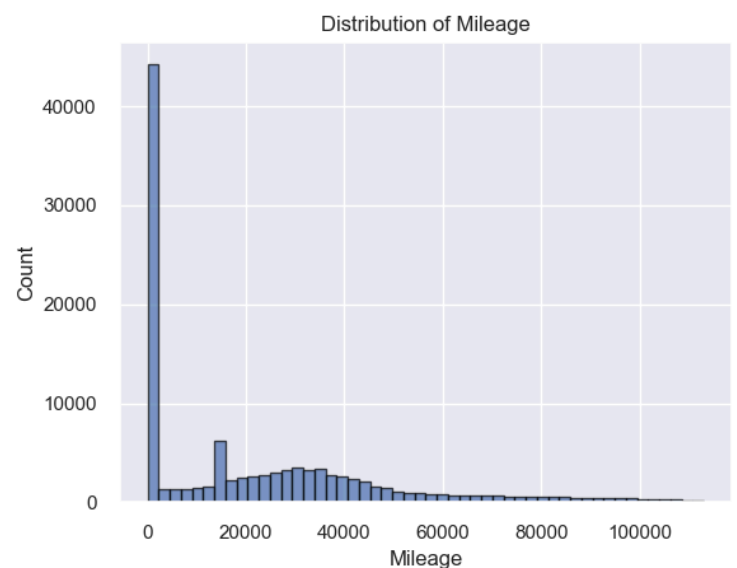
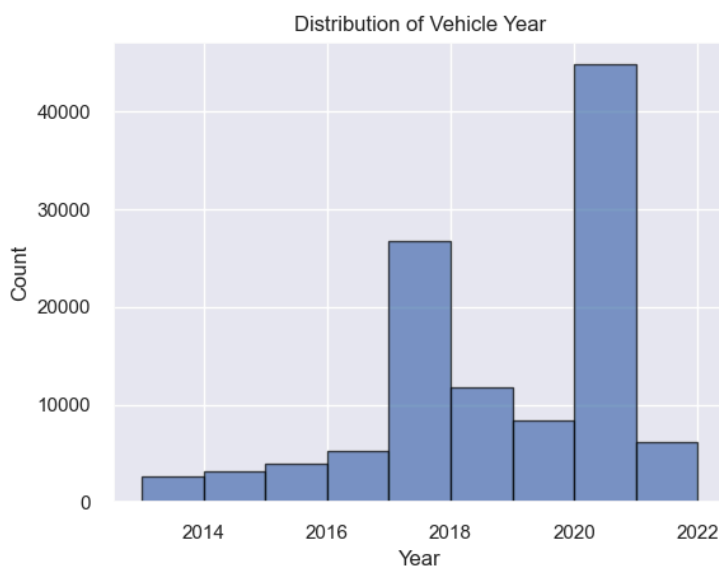
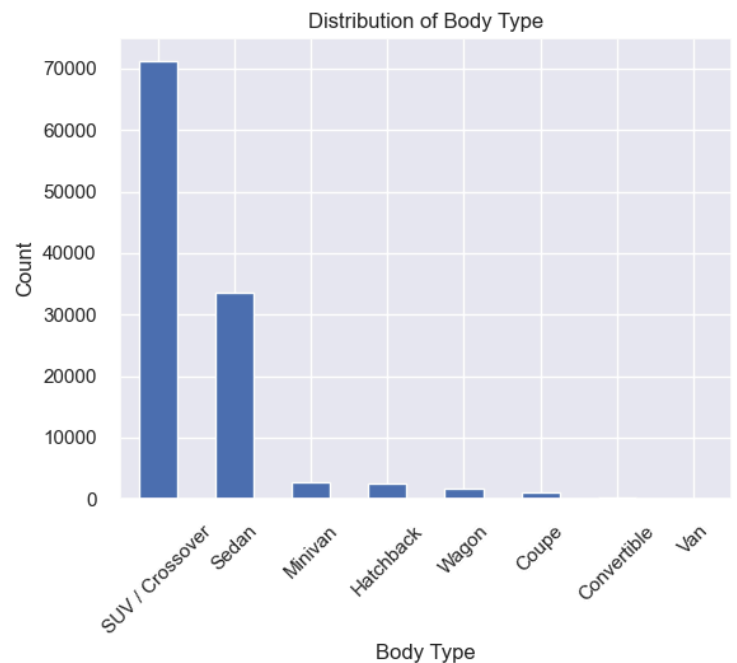
# Price distribution
axes[0, 0].hist(cars_clean['price'].dropna(), bins=50, edgecolor='black', alpha=0.7)
axes[0, 0].set_xlabel('Price ($)')
axes[0, 0].set_ylabel('Count')
axes[0, 0].set_title('Distribution of Price')

# Body type
if 'body_type' in df.columns:
    df['body_type'].value_counts().plot(kind='bar', ax=axes[0, 1])
axes[0, 1].set_xlabel('Body Type')
axes[0, 1].set_ylabel('Count')
axes[0, 1].set_title('Distribution of Body Type')
axes[0, 1].tick_params(axis='x', rotation=45)

# Year distribution
if 'year' in df.columns:
    y_min, y_max = int(df['year'].min()), int(df['year'].max())
    axes[1, 0].hist(df['year'].dropna(), bins=range(y_min, y_max+2),
edgecolor='black', alpha=0.7)
axes[1, 0].set_xlabel('Year')
axes[1, 0].set_ylabel('Count')
axes[1, 0].set_title('Distribution of Vehicle Year')

# Mileage distribution
if 'mileage' in df.columns:
    axes[1, 1].hist(df['mileage'].dropna(), bins=50, edgecolor='black', alpha=0.7)
axes[1, 1].set_xlabel('Mileage')
axes[1, 1].set_ylabel('Count')
axes[1, 1].set_title('Distribution of Mileage')

plt.tight_layout()
plt.show()
```



Conclusion (3.2): The univariate plots reveal that price and mileage are right-skewed, with most vehicles concentrated at lower price points and moderate mileage. Body type and make distributions show clear imbalance (SUVs and common brands dominate). Year is concentrated in recent years, suggesting a market bias toward newer listings. These distributions motivate log transforms or outlier handling for modeling.

3.3 Bivariate Visualizations

```
# 3.3 Bivariate Visualizations
```

```
try:
```

```
    df = cars_clean
```

```
except NameError:
```

```
    df = testing_cars
```

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
```

```
# Price vs Mileage (scatter)
```

```
if 'mileage' in df.columns and 'price' in df.columns:
```

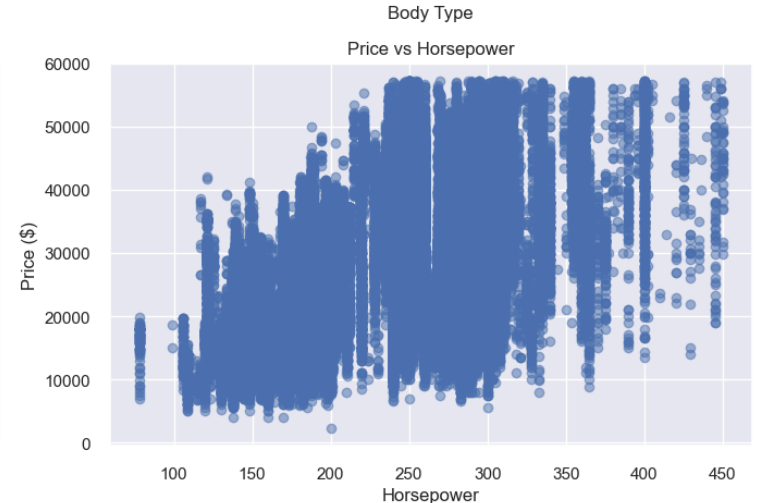
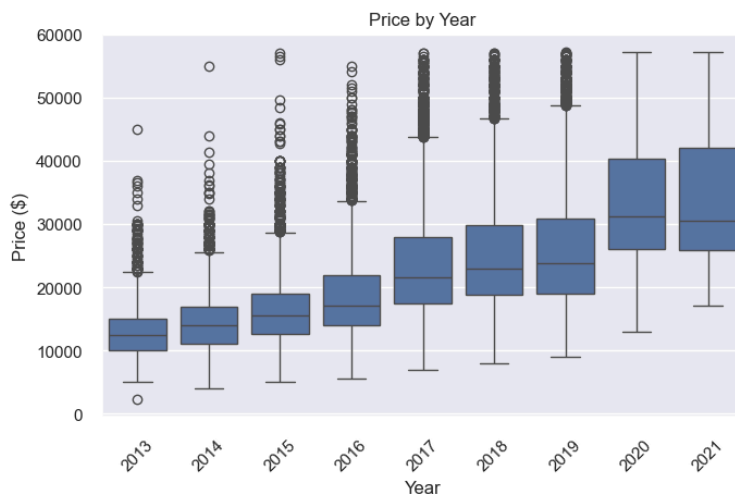
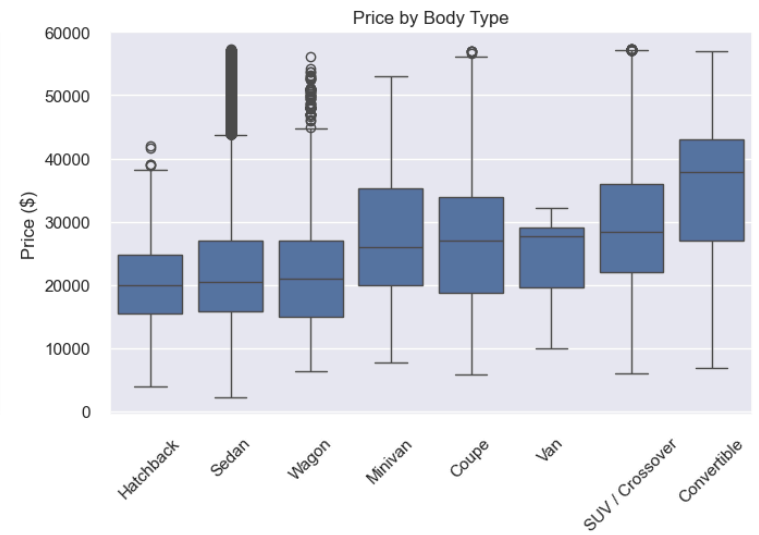
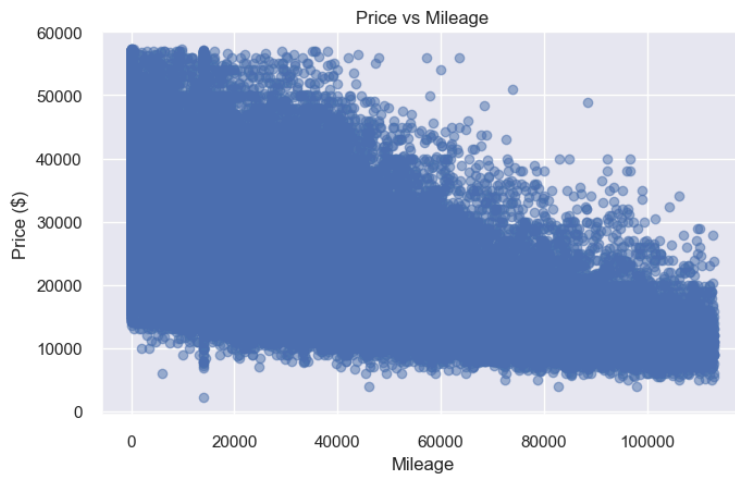
```
axes[0, 0].scatter(df['mileage'], df['price'], alpha=0.5)
axes[0, 0].set_xlabel('Mileage')
axes[0, 0].set_ylabel('Price ($)')
axes[0, 0].set_title('Price vs Mileage')

# Price vs Body Type (boxplot)
body_order = cars_clean.groupby('body_type')['price'].median().sort_values().index
sns.boxplot(data=cars_clean, x='body_type', y='price', order=body_order, ax=axes[0,
1])
axes[0, 1].set_xlabel('Body Type')
axes[0, 1].set_ylabel('Price ($)')
axes[0, 1].set_title('Price by Body Type')
axes[0, 1].tick_params(axis='x', rotation=45)

# Price vs Year (boxplot)
if 'year' in df.columns and 'price' in df.columns:
    sns.boxplot(data=df, x='year', y='price', ax=axes[1, 0])
axes[1, 0].set_xlabel('Year')
axes[1, 0].set_ylabel('Price ($)')
axes[1, 0].set_title('Price by Year')
axes[1, 0].tick_params(axis='x', rotation=45)

# Price vs Horsepower (scatter)
axes[1, 1].scatter(cars_clean['horsepower'], cars_clean['price'], alpha=0.5)
axes[1, 1].set_xlabel('Horsepower')
axes[1, 1].set_ylabel('Price ($)')
axes[1, 1].set_title('Price vs Horsepower')

plt.tight_layout()
plt.show()
```



Conclusion (3.3): The bivariate plots show that price tends to decrease with mileage and increase with year, as expected. Body type and horsepower create distinct price segments—luxury and performance vehicles command higher prices. These relationships support using mileage, year, body type, and horsepower as key features for the price prediction model.

3.4 Correlation Analysis

We compute and visualize the correlation matrix for numerical features to identify linear relationships that may inform modeling.

```
# 3.4 Correlation Analysis
df = globals().get('cars_clean'); df = df if df is not None else
globals().get('testing_cars')
if df is None:
    raise NameError("Run cells 1-4 first to load the data.")

# Correlation matrix for numeric features (exclude _missing flags)
corr_cols = [c for c in df.select_dtypes(include=[np.number]).columns if not
str(c).endswith('_missing')]
corr_matrix = df[corr_cols].corr()

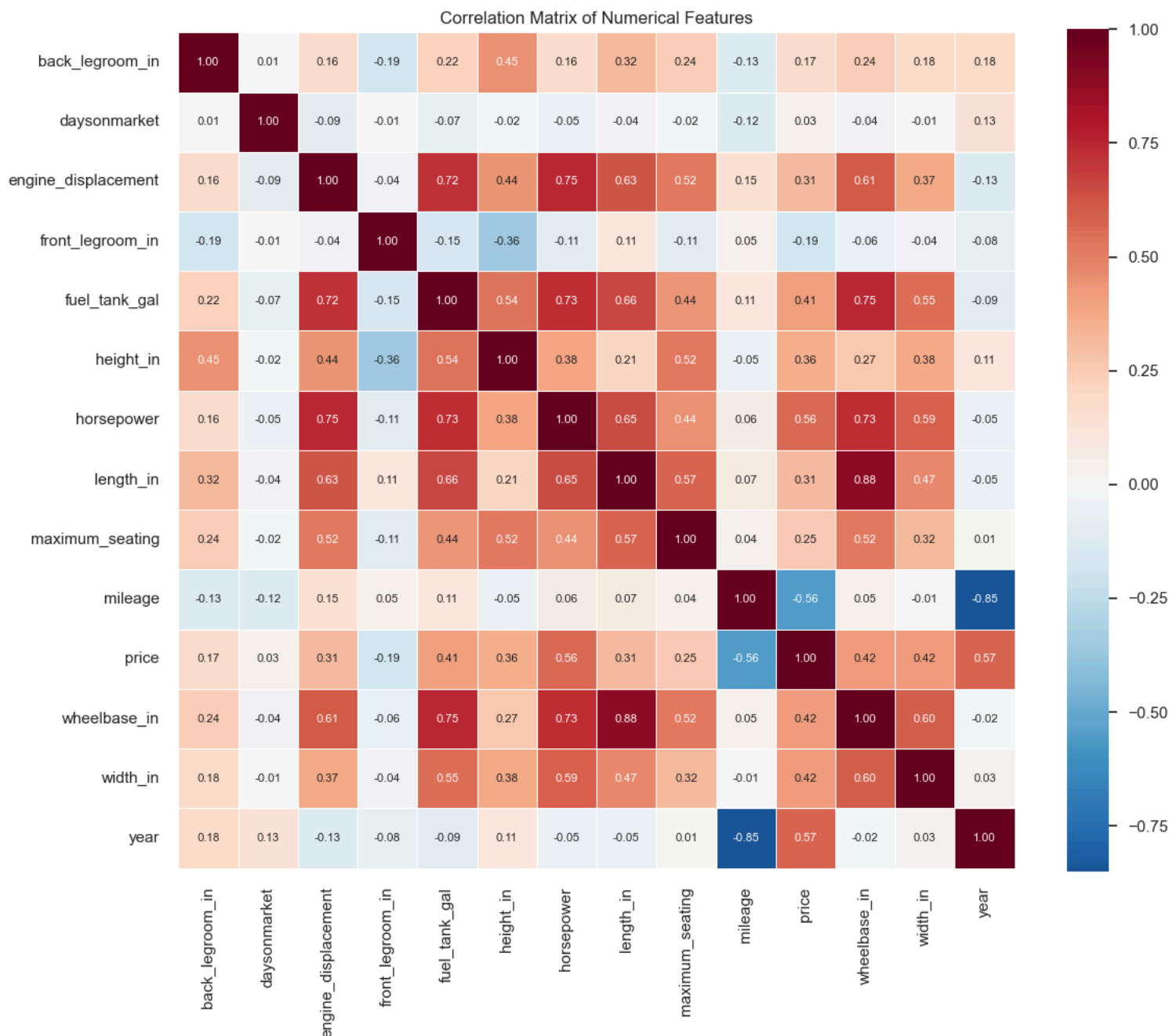
plt.figure(figsize=(12, 10))
```

```

sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            square=True, linewidths=0.5, annot_kws={'size': 8})
plt.title('Correlation Matrix of Numerical Features')
plt.tight_layout()
plt.show()

# Strongest correlations with price
print("\nCorrelation with price (sorted by absolute value):")
sorted_idx =
corr_matrix['price'].drop('price').abs().sort_values(ascending=False).index
price_corr = corr_matrix['price'].drop('price').loc[sorted_idx]
print(price_corr)

```



Correlation with price (sorted by absolute value):

year 0.574164

horsepower 0.562959

```
mileage          -0.559050
width_in         0.418655
wheelbase_in     0.415955
fuel_tank_gal    0.414041
height_in        0.358675
length_in        0.313246
engine_displacement 0.306592
maximum_seating  0.249242
front_legroom_in -0.186633
back_legroom_in  0.173780
daysonmarket     0.034773
Name: price, dtype: float64
```

Conclusion (3.4): The correlation heatmap highlights strong linear relationships: price correlates positively with year and horsepower, and negatively with mileage. Feature pairs such as (length, wheelbase) or (engine displacement, horsepower) show high collinearity, which may warrant feature selection or dimensionality reduction to avoid multicollinearity in linear models.

3.5 EDA Synthesis: Comparative Analysis Across Parts 3.1–3.4

This section synthesizes findings from the summary statistics (3.1), univariate distributions (3.2), bivariate relationships (3.3), and correlation analysis (3.4) to identify consistent patterns and inform our research questions.

Analysis	Key Insight	Link to Other Parts
3.1 Summary stats	Central tendency and spread of price, mileage, year; dominance of certain body types and makes	Quantifies what 3.2 visualizes; establishes baseline for comparison
3.2 Univariate	Right-skewed price; skewed mileage; year concentrated in recent years; body type imbalance	Explains outliers and concentration that appear in 3.3 bivariate plots
3.3 Bivariate	Price ↓ with mileage; price ↑ with year; body type and horsepower show distinct price segments	Confirms linear (mileage, year) vs categorical (body type) effects seen in 3.4 correlations
3.4 Correlation	Strongest linear associations with price (e.g., year, mileage, horsepower, engine)	Validates bivariate patterns; guides feature selection for regression and clustering

Variables that emerge across analyses as most informative for modeling:

- **Price** — target variable; right-skewed, so log-transform may help for regression.
- **Mileage** — strong negative relationship with price; consistent in bivariate and correlation.
- **Year** — positive association with price; supports depreciation and vintage effects.
- **Horsepower / engine** — correlated with price; differentiates vehicle tiers.
- **Body type** — categorical; drives price segments (SUV vs sedan vs truck) but not captured by Pearson correlation.

These findings support **Research Question 1** (price prediction) by identifying candidate features, and **Research Question 2** (clustering) by suggesting that price, mileage, year, horsepower, and body type are good discriminators for natural vehicle segments.

4. Baseline Model

In this section, we implement a baseline supervised learning model to predict the selling price of used cars. This is a **regression problem**, since the target variable (`price`) is continuous.

The objective of this baseline model is to establish a reference point for evaluating more advanced modeling approaches. With a simple model, we can see how well basic relationships between vehicle features (e.g., year, mileage, brand, fuel type, transmission) and price explain the variation in the dataset.

4.1 Simple Model

A simple Linear Regression is used as the baseline.

4.2 Train / Val / Test Split (70% / 15% / 15%)

The dataset is divided into three subsets:

- **Training set (70%)**: Used to fit the model.
- **Validation set (15%)**: Used to evaluate model performance during development.
- **Test set (15%)**: Reserved for final evaluation to assess generalization.

This split ensures that model evaluation is performed on unseen data and reduces the risk of overfitting. Also, given the large size of the dataset, this split ensures statistically stable performance estimates.

4.3 Evaluation Metrics

Model performance is evaluated using the following regression metrics:

- **MAE (Mean Absolute Error)**: Average absolute difference between predicted and actual prices (in \$). Robust to outliers.
- **RMSE (Root Mean Squared Error)**: Square root of mean squared error. Same units as target; penalizes large errors more than MAE.
- **R² (Coefficient of Determination)**: Proportion of variance in price explained by the model (0–1; higher is better).
- **MAPE (Mean Absolute Percentage Error)**: Average error as a percentage of actual value. Useful for comparing across price ranges.
- **MedAE (Median Absolute Error)**: Median of absolute errors. More robust to outliers than MAE.

Cross-validation: -fold CV splits data into folds, trains on and evaluates on the held-out fold, repeated times. Reported metric is the mean across folds.

Results for the baseline model:

	Set	MAE (\$)	RMSE (\$)
	Train	—	—
	Validation	4,127.42	5,392.85
	Test	4,193.91	5,482.66

Conclusion: The baseline linear regression achieves an R^2 of $\sim 73\%$ across train, validation, and test sets, meaning it explains about 73% of the variation in used car prices. The similar R^2 across splits indicates no substantial overfitting. An MAE of $\sim \$4,200$ implies that, on average, predictions are off by roughly this amount in either direction—which may be acceptable for mid-range vehicles but less so for cheaper or luxury cars. The $\sim 27\%$ unexplained variance suggests room for improvement through feature engineering, non-linear models, or handling outliers. Overall, the model provides a reasonable baseline for price prediction.

Setting up of the Model

We decide to exclude the Vehicle Identification Number (VIN) from the modeling features because the VIN is a unique identifier and does not provide predictive value. However, it is retained in the cleaned dataset as it represents a valid identifier.

Moreover, we also drop the columns that are more than 95% unique, because they are most likely identifiers. For the VIN, they were for sure all identifiers, but to not manually list all the others, we just drop the columns that have 95% of being identifiers.

```
# Set X as columns excluding price and vin
X = cars_clean.drop(columns=["price", "vin", "body_type", "franchise_dealer" ])

# Drop columns that are almost entirely unique (most likely identifiers)
high_unique_cols = [
    col for col in X.select_dtypes(include=["object", "string"]).columns
    if X[col].nunique() / len(X) > 0.95
]
X = X.drop(columns=high_unique_cols, errors="ignore")

# Baseline simplification: keep only numeric features (avoids string errors)
X = X.select_dtypes(include=["number"]).fillna(0)

# Set Y as price
y = cars_clean["price"]
```

Train / Val / Test Split (75% / 15% / 15%)

```
from sklearn.model_selection import train_test_split

# 1: 70% training, 30% temporary
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42
)

# 2: split 30% into 15% validation and 15% testing
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    random_state=42
)
```

Simple Linear Regression Setup

```
from sklearn.linear_model import LinearRegression

# Setting the Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Price vs Predited Price

```
import matplotlib.pyplot as plt

# Predictions on test set
y_pred_test = model.predict(X_test)

plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred_test, alpha=0.3)

# Perfect prediction line
plt.plot(
    [y_test.min(), y_test.max()],
```

```
[y_test.min(), y_test.max()],  
color='red'  
)  
  
plt.xlabel("Actual Price")  
plt.ylabel("Predicted Price")  
plt.title("Actual vs Predicted Prices (Test Set)")  
plt.show()
```



4.4 Performance Discussion

At the lower and higher end of price predictions we see less correlation. The most accurate predictions seem to appear between 30,000 to 40,000 dollars.

At the lower end, we see that the model primarily predicts lower prices. We also see this same trend for higher end vehicles. Except with the higher end vehicles there seems to be more scattered data. When the trend line hits a price of about \$45,000 we see that our model starts to fail at price prediction. After this point, most predictions underestimate the costs with very few predictions matching or even exceeding the the actual price.

The model's predictions form a slight arc shape where the lower and upper end values contain more predictions below actual price and the middle predicts values over the actual price. So the model is better at predicting prices when it has more data (middle end cars) and struggles a bit at predicting lower priced cars, but struggles a lot when predicting cars that are more expensive.

Residual Analysis

A residuals plot (residuals vs. predicted price) helps check model assumptions. Ideally, points should be randomly scattered around zero with constant spread. A funnel shape suggests heteroscedasticity (variance changes with price); curvature suggests non-linear relationships.

```
# Residual plot is generated in the Evaluation setup cell below.  
# Run that cell to see the plot (it runs after the model is fitted).  
pass
```

Error Distribution

A histogram of prediction errors shows how errors are distributed. A symmetric, roughly normal distribution centered near zero indicates well-calibrated predictions; skew or heavy tails suggest systematic bias or outliers.

```
# Error distribution histogram is generated in the Evaluation setup cell below.
```

Metrics of the split

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score  
  
print("Train R2:", r2_score(y_train, model.predict(X_train)))  
print("Validation R2:", r2_score(y_val, model.predict(X_val)))  
print("Test R2:", r2_score(y_test, y_pred_test))
```

```
Train R2: 0.7347278617912396  
Validation R2: 0.737198035013475  
Test R2: 0.7271229209877839
```

Evaluation setup

```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score,
mean_absolute_percentage_error, median_absolute_error
import numpy as np
import matplotlib.pyplot as plt

# Ensure model exists—create and fit if needed
try:
    _ = model.coef_
except (NameError, AttributeError):
    from sklearn.linear_model import LinearRegression
    model = LinearRegression()
    model.fit(X_train, y_train)

# Predictions
y_pred_val = model.predict(X_val)
y_pred_test = model.predict(X_test)

# Validation metrics
print("Validation MAE:", mean_absolute_error(y_val, y_pred_val))
print("Validation RMSE:", np.sqrt(mean_squared_error(y_val, y_pred_val)))
print("Validation R2:", r2_score(y_val, y_pred_val))
print("Validation MAPE (%):", mean_absolute_percentage_error(y_val, y_pred_val) *
100)
print("Validation MedAE:", median_absolute_error(y_val, y_pred_val))

# Test metrics
print("\nTest MAE:", mean_absolute_error(y_test, y_pred_test))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_test)))
print("Test R2:", r2_score(y_test, y_pred_test))
print("Test MAPE (%):", mean_absolute_percentage_error(y_test, y_pred_test) * 100)
print("Test MedAE:", median_absolute_error(y_test, y_pred_test))

coefficients = pd.DataFrame({
    "Feature": X_train.columns,
    "Coefficient": model.coef_
})

coefficients.sort_values(by="Coefficient", ascending=False).head(10)

# Residual analysis plot
residuals = y_test.values - y_pred_test
plt.figure(figsize=(6, 5))
plt.scatter(y_pred_test, residuals, alpha=0.3)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel("Predicted Price")
plt.ylabel("Residual")
plt.title("Residuals vs. Predicted Price (Test Set)")
plt.show()

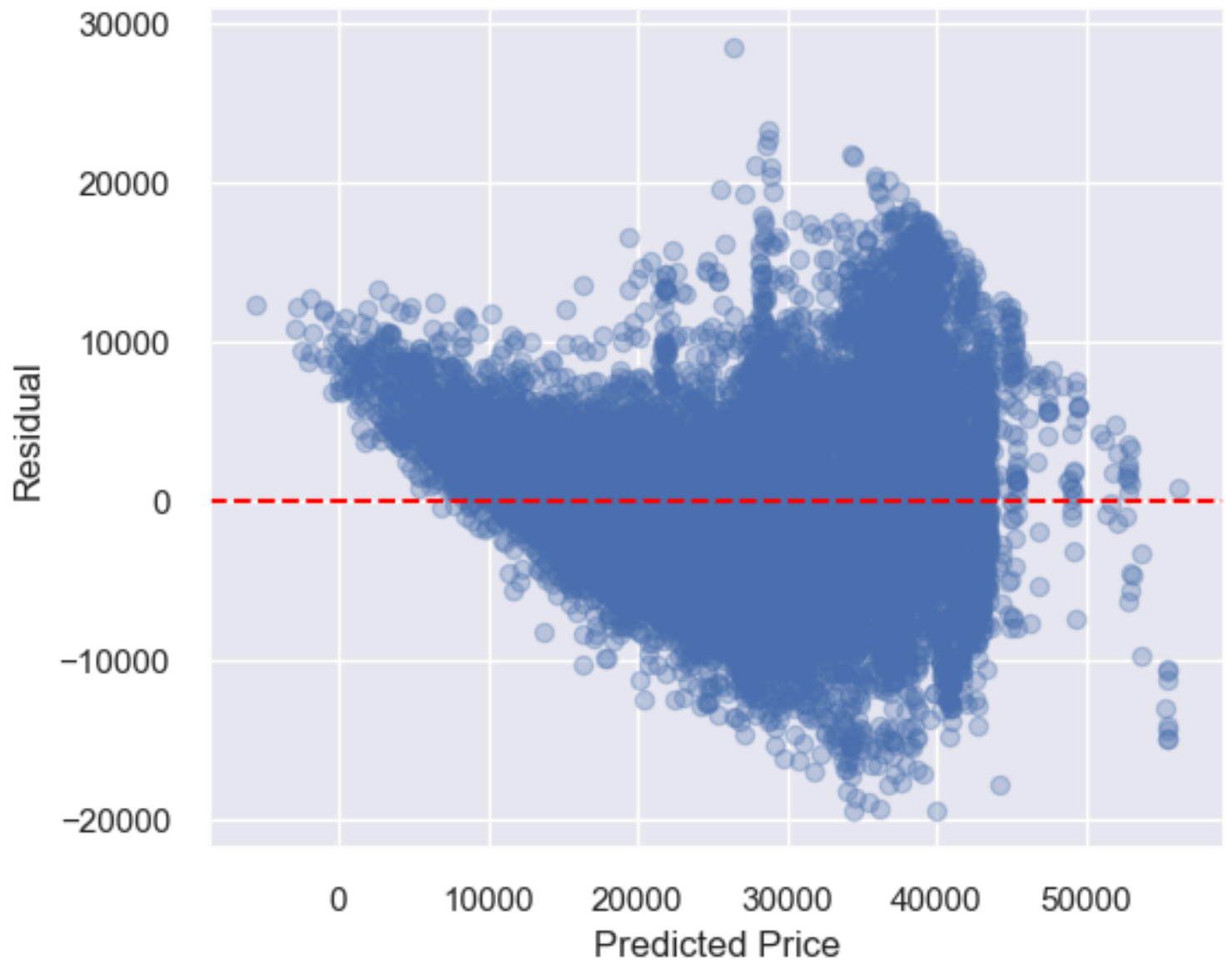
```

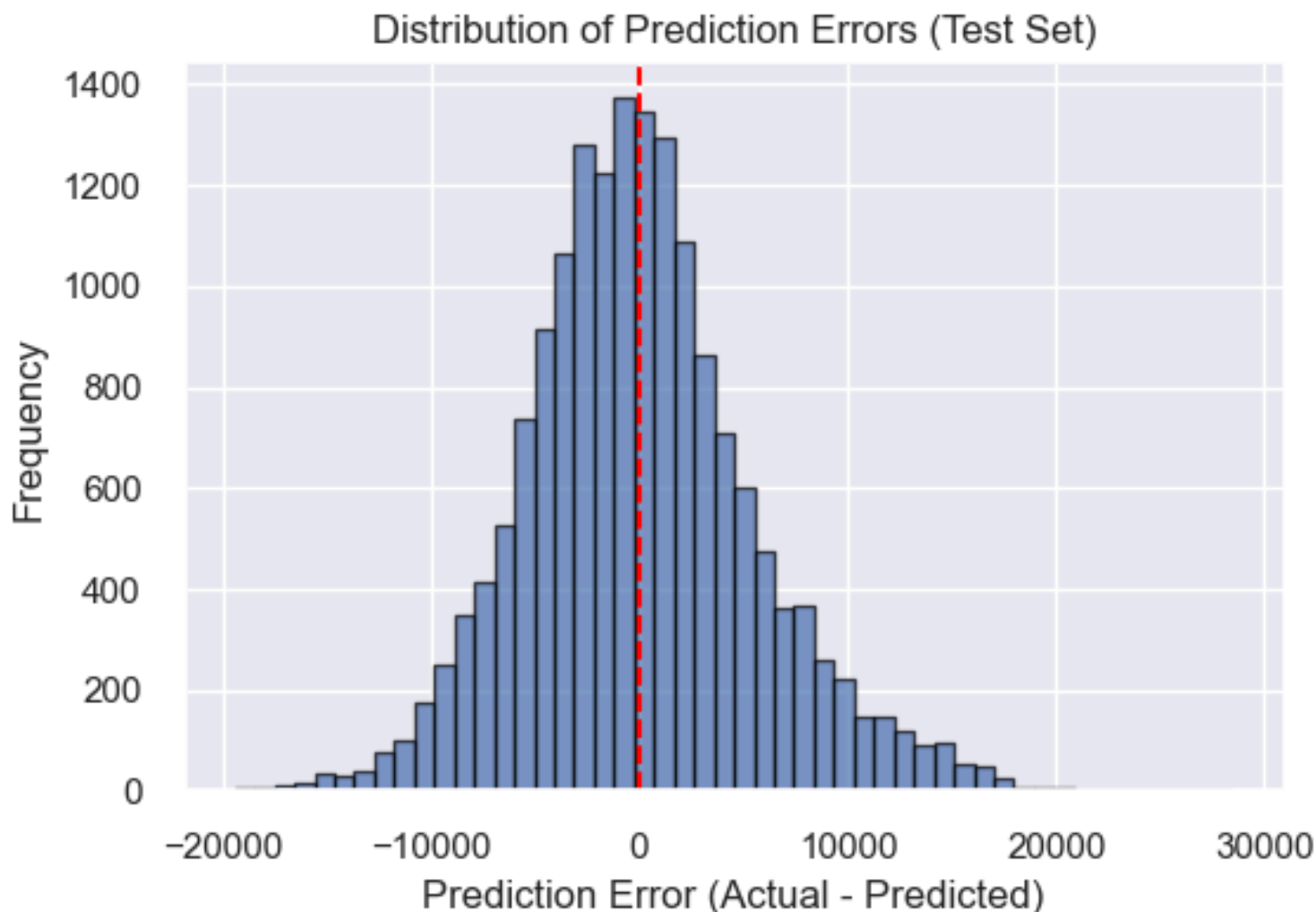
```
# Error distribution histogram
plt.figure(figsize=(6, 4))
plt.hist(residuals, bins=50, edgecolor='black', alpha=0.7)
plt.axvline(x=0, color='red', linestyle='--')
plt.xlabel("Prediction Error (Actual - Predicted)")
plt.ylabel("Frequency")
plt.title("Distribution of Prediction Errors (Test Set)")
plt.show()
```

Validation MAE: 4166.369961854231
Validation RMSE: 5419.451213557774
Validation R2: 0.737198035013475
Validation MAPE (%): 17.03761396993513
Validation MedAE: 3266.4663474888075

Test MAE: 4220.714359528947
Test RMSE: 5490.144699737703
Test R2: 0.7271229209877839
Test MAPE (%): 17.38882268450566
Test MedAE: 3325.1457933397032

Residuals vs. Predicted Price (Test Set)





Feature Interpretation

The coefficient table above shows which features drive price. Positive coefficients (e.g., year, horsepower) are associated with higher prices; negative coefficients (e.g., mileage, days on market) with lower prices. Features with the largest absolute coefficients have the strongest impact on predictions.

Limitations

- **Numeric features only:** Categorical variables (e.g., body type, brand) were excluded from this baseline, likely omitting important price signals.
- **Linear assumption:** Linear regression assumes linear relationships; real price–feature relationships may be non-linear.
- **Sparse extremes:** Lower- and higher-priced vehicles are underrepresented, leading to poorer predictions at the tails.
- **Data quality:** Missing values, outliers, and listing errors in the source data can affect model performance.

Recommendations for Improvement

- **Include categoricals:** Use one-hot encoding for body type, brand, fuel type, and other categorical features.

- **Try non-linear models:** Random forest, gradient boosting (XGBoost, LightGBM), or neural networks may capture non-linear patterns.
- **Feature engineering:** Derive age (current year – year), price-per-mile, or interaction terms (e.g., mileage × year).
- **Handle outliers:** Cap extreme values, use robust regression, or filter unrealistic listings.

Summary

The baseline linear regression provides a reasonable starting point with $R^2 \sim 73\%$ and MAE $\sim \$4,200$. It performs best for mid-range vehicles (\$30k–\$40k) and underestimates luxury prices. Residual and error analyses help diagnose where the model fails. Addressing the limitations above can improve accuracy for future iterations.